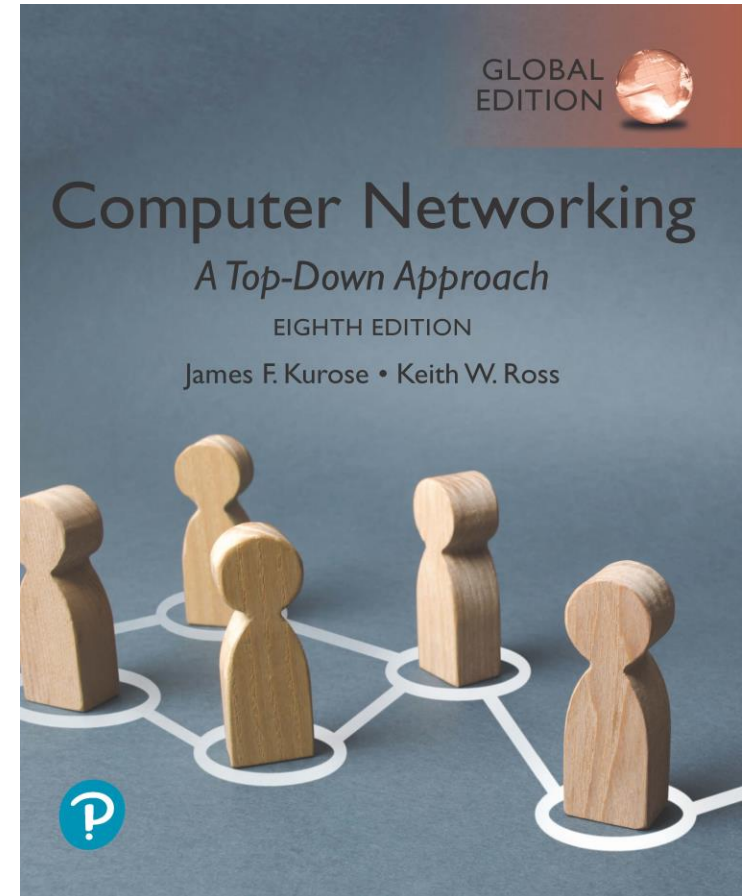


Chapter 8

Security



Computer Networking: A Top-Down Approach

8th edition, Global Edition

Jim Kurose, Keith Ross

Copyright © 2022 Pearson Education Ltd

Chapter 8 outline

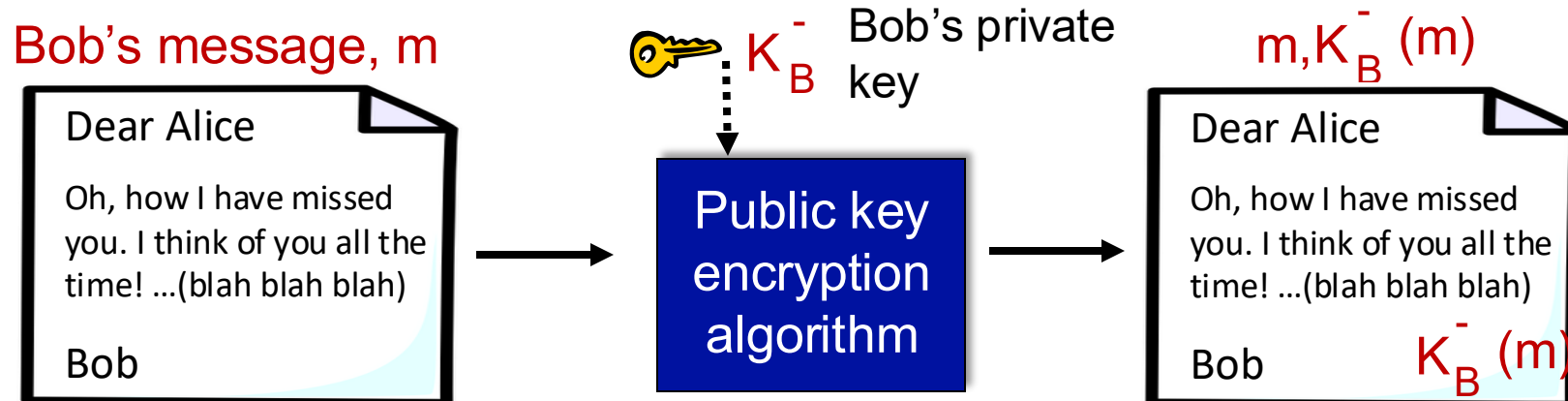
- What is network security?
- Principles of cryptography
- **message integrity**
- Securing e-mail
- Securing TCP connections: TLS
- Network layer security: IPsec
- Security in wireless and mobile networks
- Operational security: firewalls and IDS



Digital signatures

cryptographic technique analogous to hand-written signatures:

- sender (Bob) **digitally signs** document: he is document owner/creator.
- *verifiable, nonforgeable*: recipient (Alice) can prove to someone that Bob, and no one else (including Alice), must have signed document
- **simple digital signature for message m :**
 - Bob signs m by encrypting with his private key K_B , creating “signed” message, $K_B^-(m)$



Digital signatures

- suppose Alice receives msg m , with signature: $m, K_B^-(m)$
- Alice verifies m signed by Bob by applying Bob's public key K_B^+ to $K_B^-(m)$ then checks $K_B^+(K_B^-(m)) = m$.
- If $K_B(K_B(m)) = m$, whoever signed m must have used Bob's private key

Alice thus verifies that:

- Bob signed m
- no one else signed m
- Bob signed m and not m'

non-repudiation:

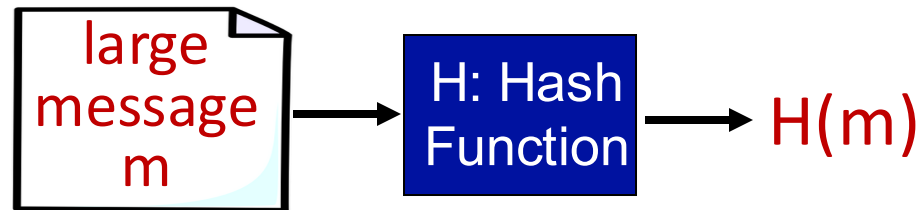
- ✓ Alice can take m , and signature $K_B^-(m)$ to court and prove that Bob signed m

Message digests

computationally expensive to public-key-encrypt long messages

goal: fixed-length, easy- to-compute digital “fingerprint”

- apply hash function H to m , get fixed size **message digest, $H(m)$**



Hash function properties:

- many-to-1
- produces fixed-size msg digest (fingerprint)
- given message digest x , computationally infeasible to find m such that $x = H(m)$

Hashes, also called message digests, are designed to produce a fixed-size hash value for variable-sized (and normally much larger) inputs. Since there are many more possible inputs than possible outputs, it is apparent that many inputs must produce the same output. But a good cryptographic hash algorithm makes it computationally very difficult to find even two different inputs that produce the same output.

So, if you are given a specific hash output and have an input that produces that hash value, you can be reasonably certain that that input is the one that produced the given hash output. This justifies the use of hash values to verify that data has not been corrupted or tampered with, and, in particular, the use of hash values for producing public key signatures.

Internet checksum: poor crypto hash function

Internet checksum has some properties of hash function:

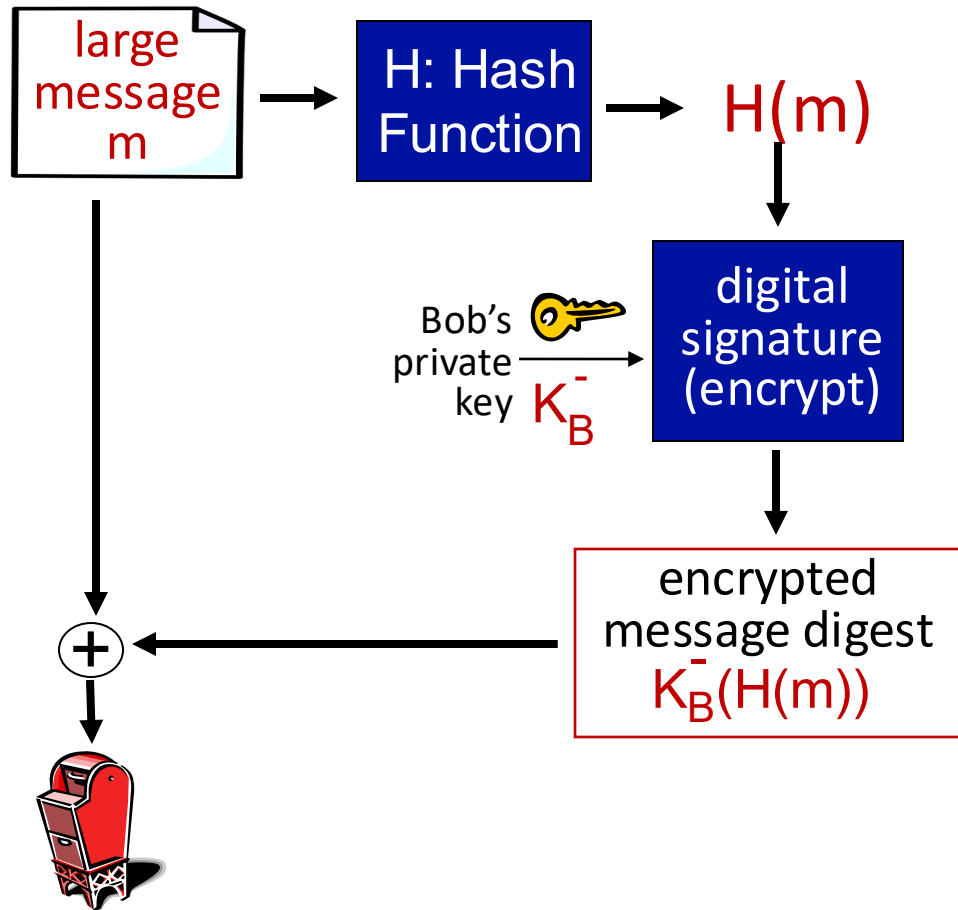
- produces fixed length digest (16-bit sum) of message
- is many-to-one

but given message with given hash value, it is easy to find another message with same hash value:

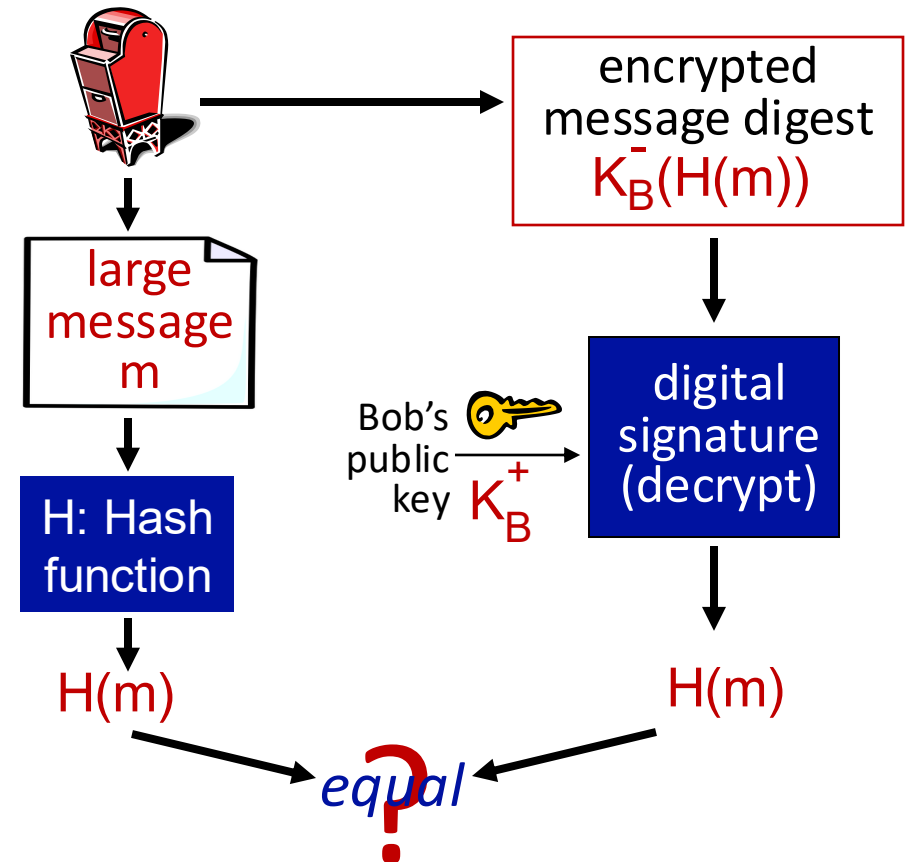
<u>message</u>	<u>ASCII format</u>		<u>message</u>	<u>ASCII format</u>
I O U 1	49 4F 55 31		I O U <u>9</u>	49 4F 55 <u>39</u>
0 0 . 9	30 30 2E 39		0 0 . <u>1</u>	30 30 2E <u>31</u>
9 B O B	39 42 D2 42		9 B O B	39 42 D2 42
B2 C1 D2 AC		<i>different messages</i> <i>but identical checksums!</i>	B2 C1 D2 AC	

Digital signature = signed message digest

Bob sends digitally signed message:



Alice verifies signature, integrity of digitally signed message:



Hash function algorithms

- **MD5** hash function widely used (RFC 1321)
 - computes 128-bit message digest in 4-step process.
 - arbitrary 128-bit string x , appears difficult to construct msg m whose MD5 hash is equal to x
- **SHA-1** is also used
 - US standard [NIST, FIPS PUB 180-1]
 - 160-bit message digest

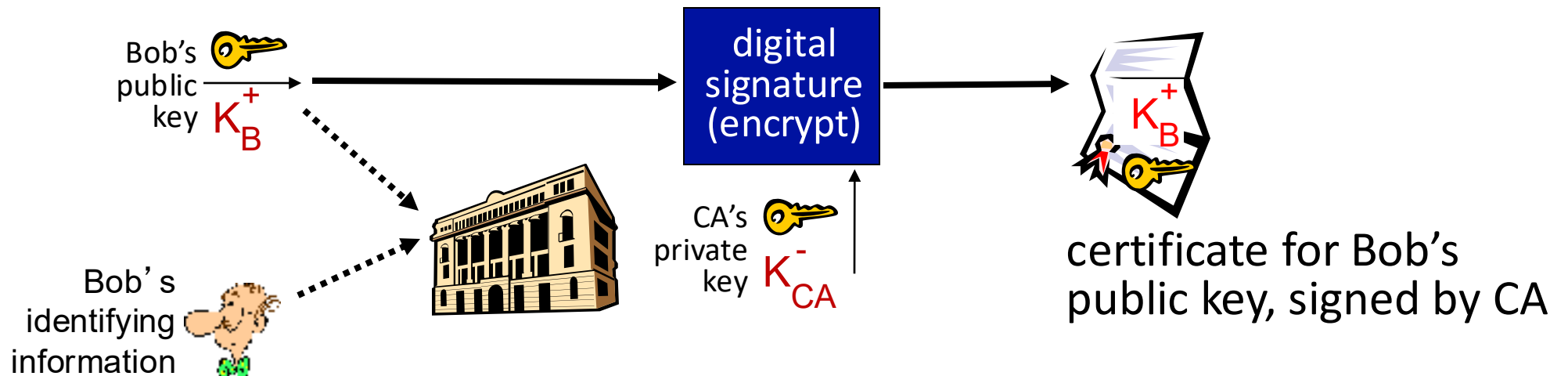
Need for **certified** public keys

- motivation: Trudy plays pizza prank on Bob
 - Trudy creates e-mail order:
Dear Pizza Store, Please deliver to me four pepperoni pizzas. Thank you, Bob
 - Trudy signs order with her private key
 - Trudy sends order to Pizza Store
 - Trudy sends to Pizza Store her public key, but says it's Bob's public key
 - Pizza Store verifies signature; then delivers four pepperoni pizzas to Bob
 - Bob doesn't even like pepperoni



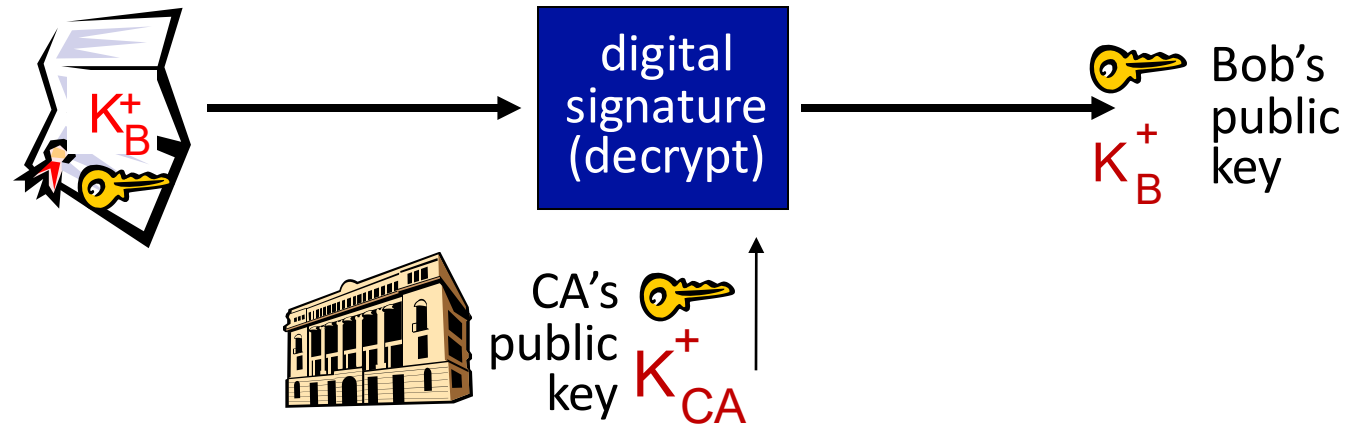
Public key Certification Authorities (CA)

- **certification authority (CA)**: binds public key to particular entity, E
- **entity** (person, website, router) registers its public key with CA and provides “proof of identity” to CA
 - CA creates certificate binding identity E to E’s public key
 - certificate containing E’s public key digitally signed by CA: CA says “this is E’s public key”



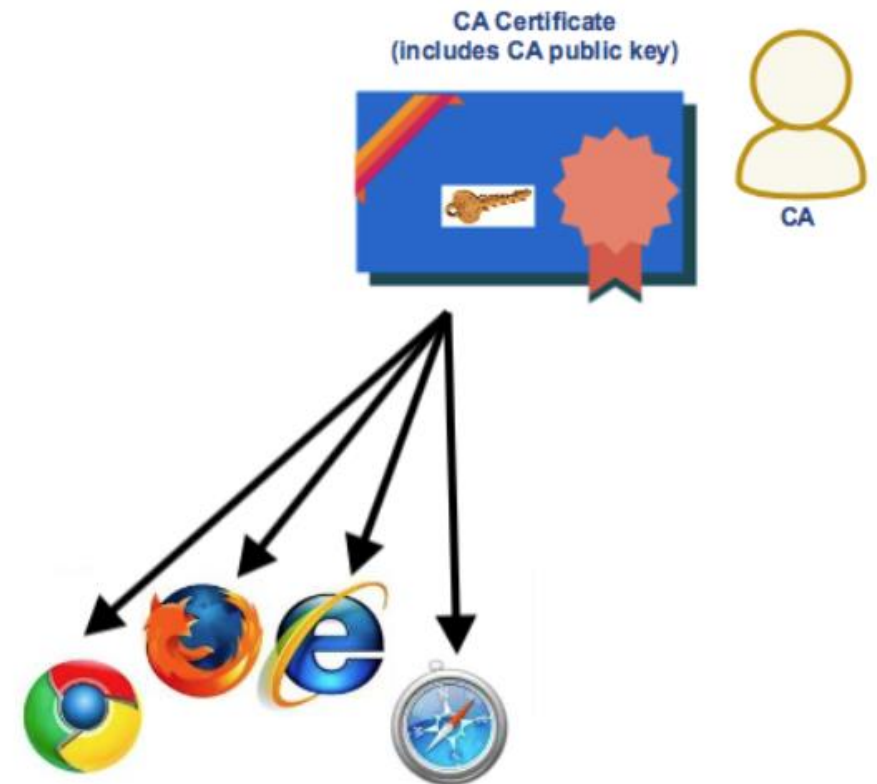
Public key Certification Authorities (CA)

- when Alice wants Bob's public key:
 - gets Bob's certificate (Bob or elsewhere)
 - if Bob is a web server you can get it at connect time
 - apply CA's public key to Bob's certificate, get Bob's public key
 - Where do I get the CA's public key ? Wait for it...



How to trust the CA ?

Before any major Web browser like Chrome, Firefox, Safari or Internet Explorer connects to your server via HTTPS, it already has in its possession a set of certificates that can be used to verify the [digital signature](#) that will be found on your server certificate. These web browser digital certificates are called CA (Certificate Authority) certificates. On these certificates is a copy of the public key of the CA who might issue (sign) your server certificate.



Major browsers come pre-installed
with CA Certificates of well-respected Certificate Authorities



Innstillinger

 **Innstillinger**

Søk i innstillingene

 Deg og Google

 Autofyll

 **Personvern og sikkerhet**

 Ytelse

 Utseende

 Søkemotor

 Ved oppstart

 Språk

 Nedlastinger

 Tilgjengelighet

 Tilbakestill innstillingene

 Utvidelser 

 Om Chrome

 Administrer sertifikater

Dine sertifikater

Tjenere

Utstedere

Andre

Du har registrerte sertifikater som identifiserer disse sertifikatinstansene

Importér

org-AC Camerfirma S.A.	▼
org-AC Camerfirma SA CIF A82743287	▼
org-ACCV	▼
org-Actalis S.p.A./03358520967	▼
org-AffirmTrust	▼
org-Agencia Catalana de Certificacio (NIF Q-0801176-I)	▼
org-Amazon	▼
org-ANF Autoridad de Certificacion	▼
org-Asseco Data Systems S.A.	▼
org-Atos	▼
org-Autoridad de Certificacion Firmaprofesional CIF A62634068	▼

Kommer ikke inn i den nye helse-appen

Flere brukere melder om problemer med Helseplattformens tjeneste.



– Dette skyldes som regel sviktende driftsrutiner. Sikkerheten til nettstedet vil ikke gå ned, sier Harald Hauknes.

Han er sikkerhetsrådgiver i Watchcom Security Group. Ifølge Hauknes har SSL-sertifikatet, eller sikkerhetssertifikatet, til Helsami gått ut.

Går god for nettstedet

– Nettleseren vil gjerne slå seg vrang når sertifikatet har gått ut, med mindre man overstyrer det. Muligheten til å overstyre er ikke til stede i mobilappen. Veldig mange brukere har mistet tilgang til tjenesten.

Sertifikatet bekrefter at nettstedet er hva det gir seg ut for å være, forklarer Hauknes.

– Det er en tjeneste fra en ekstern leverandør som går god for at et nettsted har rett til å kalle seg for eksempel Helsami.

